Project Synopsis

For

DISTRIBUTED CLOUD COMPUTE NETWORK FOR DYNAMIC RESOURCE ALLOCATION

[Submission Date]

Prepared by

Specialization	SAP ID	Name
[Specialization]	[SAP ID]	[Student Name]

School of Computer Science
UNIVERSITY OF PETROLEUM & ENERGY STUDIES,
DEHRADUN - 248007, UTTARAKHAND

Table of Contents

No.	Topic	Page No.
	Table of Contents	2
	Revision History	3
1	INTRODUCTION	4
	1.1 Purpose of the Project	4
	1.2 Target Beneficiary	4
	1.3 Project Scope	4
	1.4 References	4
2	PROJECT DESCRIPTION	5
	2.1 Reference Algorithm	5
	2.2 Data / Data Structure	5
	2.3 SWOT Analysis	6
	2.4 Project Features	6
	2.5 User Classes and Characteristics	6
	2.6 Design and Implementation Constraints	6
	2.7 Design Diagrams	7
	2.8 Assumptions and Dependencies	10
3	SYSTEM REQUIREMENTS	10
	3.1 User Interface	10
	3.2 Software Interface	10
	3.3 Database Interface	11
	3.4 Protocols	11
4	NON-FUNCTIONAL REQUIREMENTS	12
	4.1 Performance Requirements	12
	4.2 Security Requirements	12
	4.3 Software Quality Attributes	12
5	OTHER REQUIREMENTS	13
	Appendix A: Glossary	13
	Appendix B: Analysis Model	13
	Appendix C: Issues List	13

Revision History

Date	Change	Reason for Changes	Mentor Signature
20 Aug 2026	Version 1.0 - complete synopsis	Initial submission draft based on the approved project concept	

Document status: Prepared as a major-project synopsis. Personal details, submission date, mentor signature and any department-specific approval fields should be completed before submission.

1 INTRODUCTION

1.1 Purpose of the Project

Modern personal computers, university laboratories and small organizational servers often remain under-utilized for long periods, even though they contain capable multi-core CPUs and, in some cases, expensive GPUs. At the same time, students, researchers and small teams may need substantial computing power for a short data-processing, simulation, rendering or machine-learning task but cannot justify buying new hardware or maintaining a permanent cloud instance. The proposed project addresses this mismatch by pooling volunteered machines into a controlled distributed compute network. A user describes a workload and its resource needs; the platform discovers currently available nodes, selects a suitable machine and starts the workload inside an isolated container.

The purpose is not to reproduce a commercial public cloud. It is to design and evaluate a student-scale control plane that can coordinate heterogeneous and intermittently available resources. The work concentrates on dynamic discovery, multi-resource scheduling, container-based execution, monitoring and restart-based failure recovery. NIST describes resource pooling, on-demand access and measured service as defining cloud characteristics [1]; these ideas motivate the architecture, while the implementation remains deliberately limited to trusted participants and approved workload images.

1.2 Target Beneficiary

Primary beneficiaries are university students and research groups that need burst CPU/GPU capacity, laboratories seeking better use of existing workstations, and small organizations with short computational jobs. Compute providers benefit from visibility into how their contributed resources are used. Administrators gain a single dashboard for node health, scheduling decisions, failures and audit events.

1.3 Project Scope

The prototype will include a web interface, authenticated control API, resource registry, job queue, scheduler, PostgreSQL database, Redis-backed dispatch queue and a Python node agent. Linux worker nodes will report CPU, memory, storage, network and optional NVIDIA GPU telemetry. Jobs will be submitted as approved Docker images with declared limits, executed as non-root containers and returned as logs plus output artifacts. The scheduler will filter infeasible nodes, rank the remaining candidates, reserve capacity atomically and requeue a job when an assigned node becomes unavailable. Payments, arbitrary untrusted images, cross-host distributed training and production-grade billing are outside the initial scope.

1.4 References

Core sources	Supporting sources
[1] Mell and Grance, NIST SP 800-145, Cloud Computing, 2011.	[7] Verma et al., Borg Cluster Management, EuroSys, 2015.
[2] Kubernetes, Kube-scheduler and Resource Bin Packing, 2026.	[8] Burns et al., Borg, Omega, and Kubernetes, 2016.
[3] Kubernetes, GPU Scheduling and Device Plugins, 2026.	[9] NIST SP 800-190, Application Container Security, 2017.
[4] Docker, Container Resource Constraints, 2026.	[10] OWASP Docker Security Cheat Sheet, 2026.
[5] NVIDIA Container Toolkit Documentation, 2026.	[11] PostgreSQL 18, Transaction Isolation, 2026.
[6] Ghodsi et al., DRF, NSDI, 2011.	[12] OpenTelemetry Documentation, 2026.

2 PROJECT DESCRIPTION

2.1 Reference Algorithm

The scheduler follows a filter-rank-reserve-dispatch method similar to the separation of feasibility and scoring used by cluster schedulers [2]. Let job j request CPU c, memory m, storage d, GPU count g, minimum GPU memory v and a maximum execution time. A node enters the eligible set only when it is online, not drained, supports the requested runtime and has sufficient unreserved resources. For GPU jobs, the device type and available VRAM must also satisfy the request. Hard filtering prevents a high score from hiding an invalid placement.

Eligibility and ranking: E(j) = {i | freeCPUi >= cj, freeRAMi >= mj, freeDiski >= dj, GPUi satisfies (gj, vj), and node i is healthy}. For i in E(j): Score(i,j) = 0.22C + 0.18M + 0.25G + 0.12L + 0.08N + 0.10R + 0.05P. The highest-scoring node is reserved through a database transaction before dispatch.

C and M express normalized free CPU and memory; G measures GPU suitability without automatically favoring the most powerful card; L rewards lower current utilization; N reflects inverse network latency; R is a rolling reliability score based on recent heartbeats and completed attempts; and P rewards placements that leave useful resource combinations for later jobs. Values are scaled to 0-1. If no eligible node exists, the job remains queued and is reconsidered when new telemetry arrives. To reduce starvation, waiting time adds a small age bonus. When several users compete, the scheduler can apply a dominant-share penalty inspired by Dominant Resource Fairness [6], so one user cannot occupy most of both CPU and GPU capacity.

Step	Stage	Scheduler action
1	Validate	Reject malformed limits, unknown images and unsupported runtimes.
2	Filter	Remove offline, drained, incompatible or under-capacity nodes.
3	Rank	Calculate the weighted score from the latest non-stale snapshot.
4	Reserve	Lock the selected capacity and create an allocation attempt atomically.
5	Dispatch	Send a signed job command; agent applies limits and starts the container.
6	Recover	On timeout or lost heartbeat, release the reservation and requeue within retry policy.

2.2 Data / Data Structure

The project does not depend on a fixed external dataset. Its primary data is operational telemetry generated by participating nodes and job-lifecycle events produced by the control plane. Every five seconds, the agent samples CPU load, free memory, disk availability, GPU utilization/VRAM through NVIDIA Management Library when present, active containers and local queue depth. A heartbeat is sent every ten seconds; a node is marked suspect after three missed heartbeats and unavailable after sixty seconds. Job records contain an image digest, resource request, owner, priority, attempt count, timestamps, output location and terminal status. Personally identifiable workload content is neither required nor analyzed.

Secondary data consists of documented hardware characteristics and controlled benchmark workloads: CPU matrix multiplication or compression, GPU image inference, and mixed short/long jobs. Experiments will replay the same stratified workload trace against round-robin, random and proposed resource-aware scheduling. Metrics are normalized with bounded min-max scaling; one-minute rolling medians smooth transient utilization. Results will report median and 95th-percentile queue time, completion time, utilization, success rate and recovery delay. Outliers will be retained but identified using the interquartile-range rule, and policy differences will be summarized with paired bootstrap 95% confidence intervals.

2.3 SWOT Analysis

Dimension	Project-specific assessment
Strengths	Reuses idle heterogeneous hardware; resource-aware placement; container limits; measurable utilization and recovery.
Weaknesses	Central control plane; transfer overhead for large images; whole-GPU allocation; Docker is weaker isolation than a VM.
Opportunities	Campus compute pool, research inference, rendering, video processing, energy-aware scheduling and future federation.
Threats	Malicious images, node churn, driver mismatch, unreliable networks, data leakage and provider policy or licensing limits.

2.4 Project Features

- Role-based registration and authentication for compute users, providers and administrators.
- Automatic node discovery, hardware inventory, heartbeat monitoring and provider-controlled availability.
- Job submission using approved container images, explicit CPU/RAM/GPU/storage limits and timeout policy.
- Constraint filtering, weighted ranking, transactional reservation and explainable scheduling decisions.
- Container execution, live status/logs, result retrieval, cancellation, retry and node-failure requeueing.
- Network dashboard showing online nodes, reservations, utilization, queue depth, failures and audit history.

2.5 User Classes and Characteristics

User class	Characteristics and permitted actions
Compute User	Submits trusted workloads, declares requirements, monitors progress and retrieves outputs; requires no knowledge of the selected physical node.
Compute Provider	Installs the agent, approves node participation, sets availability and views local utilization; understands basic Linux/Docker operations.
Administrator	Approves nodes/images, manages accounts and policies, changes scheduler weights, investigates failures and reviews audit events.

2.6 Design and Implementation Constraints

Worker nodes will run 64-bit Linux with Docker Engine; GPU execution requires a compatible NVIDIA driver and Container Toolkit [5]. The initial network will contain approximately three to ten trusted nodes and will allocate an entire GPU device to a job rather than attempting fractional VRAM control. Container images must come from an administrator-approved registry and execute as non-root with CPU, memory, process, storage and time limits. The central server requires PostgreSQL, Redis and synchronized system clocks. Communication must tolerate variable consumer-network latency, but the system will not support MPI jobs, cross-host distributed training, live migration or transparent resume unless the workload itself writes checkpoints. REST resources will be versioned, source code will follow formatter/linter rules, and each state transition must be auditable.

2.7 Design Diagrams

The following models define the principal actors, static domain objects and behavior of the proposed system. They are logical design artifacts; individual implementation classes may be refined during development.

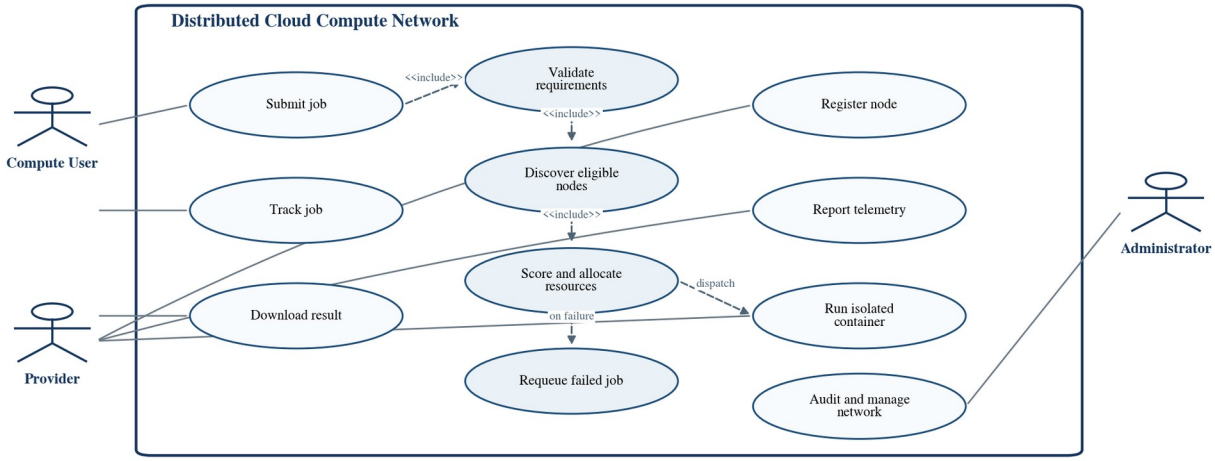


Figure 1. Level-2 use case diagram *Source: Developed by the project team.*

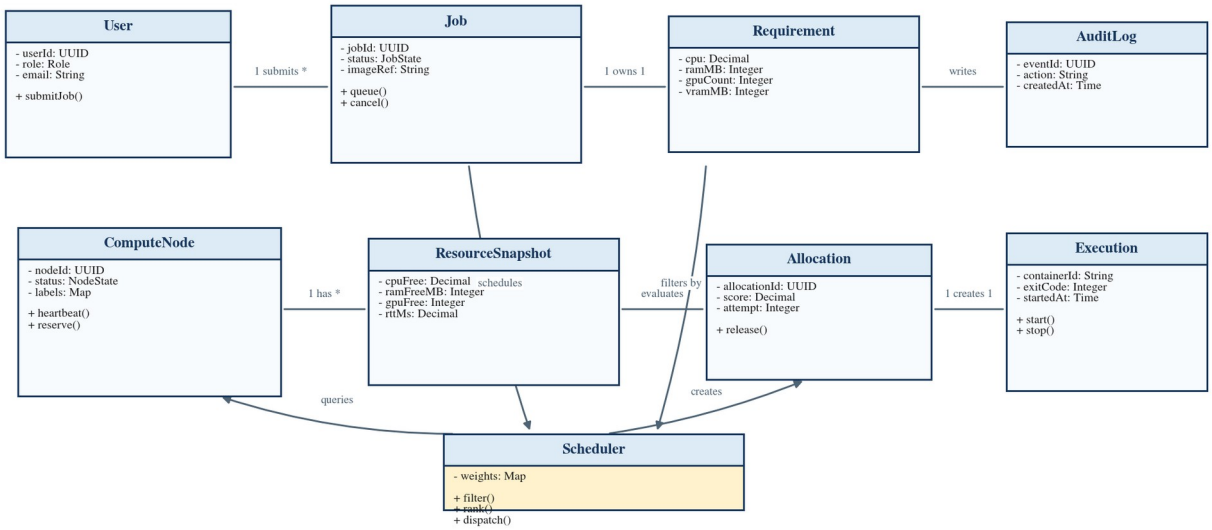


Figure 2. Core class diagram *Source: Developed by the project team.*

2.7 Design Diagrams (Continued)

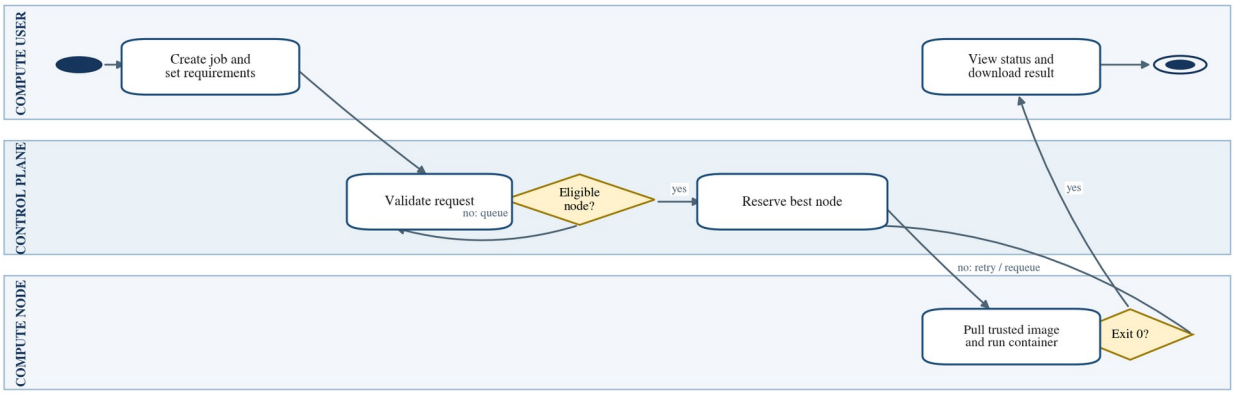


Figure 3. Job allocation activity diagram *Source: Developed by the project team.*

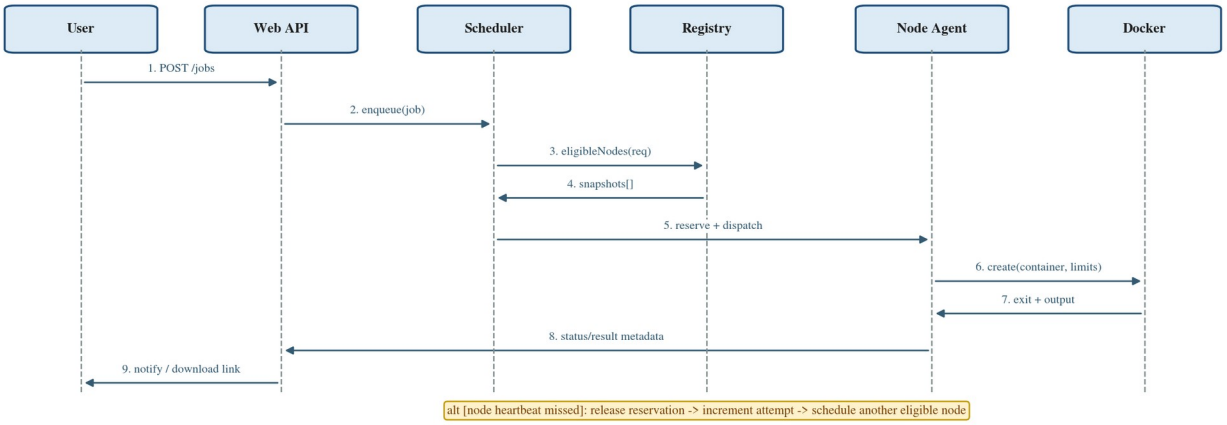


Figure 4. Successful job-attempt sequence diagram *Source: Developed by the project team.*

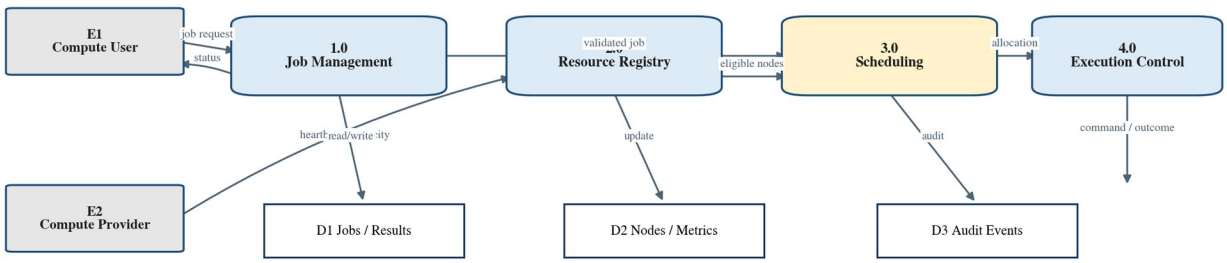


Figure 5. Level-1 data flow diagram *Source: Developed by the project team.*

2.7 Design Diagrams (Continued)

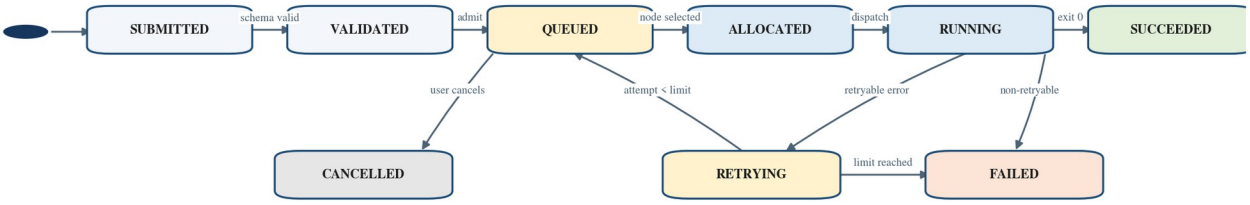


Figure 6. Job lifecycle state diagram *Source: Developed by the project team.*

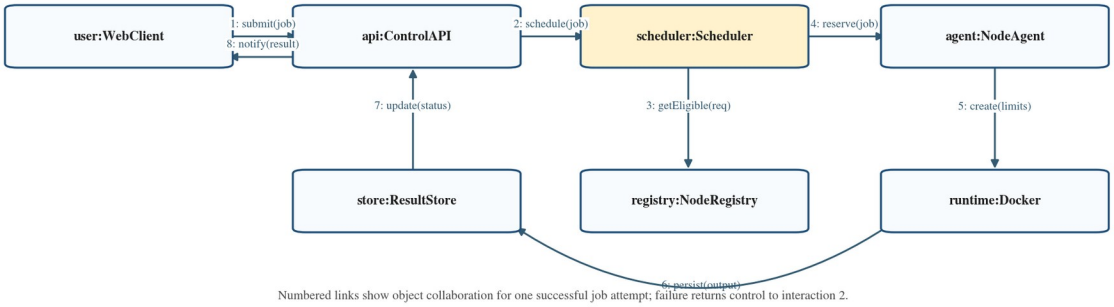


Figure 7. Object collaboration diagram *Source: Developed by the project team.*

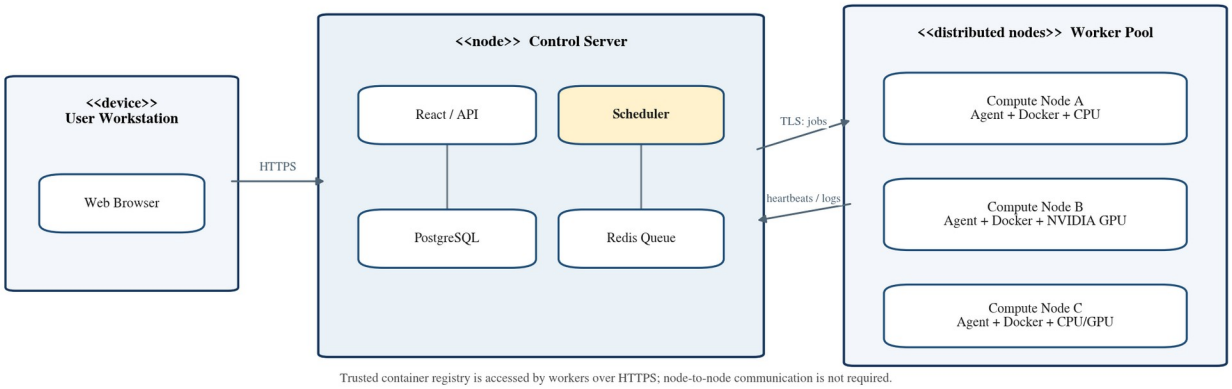


Figure 8. Deployment diagram for the prototype *Source: Developed by the project team.*

2.8 Assumptions and Dependencies

- Participants are known to the institution and workloads use administrator-approved images; the prototype is not an anonymous public marketplace.
- Nodes may join or leave, but they remain online long enough to complete most selected jobs and provide reasonably accurate telemetry.
- A stable control server, PostgreSQL database, Redis service, object storage or shared result directory and trusted container registry are available.
- GPU nodes have compatible NVIDIA drivers; CPU-only jobs remain portable across x86-64 Linux workers. Network time synchronization is enabled.
- Recovery means restart from the beginning unless the application stores a checkpoint in durable storage.

3 SYSTEM REQUIREMENTS

3.1 User Interface

The responsive web interface will expose role-specific navigation. A compute user sees a job form, requirement validation, queue/running/completed views, an allocation explanation, logs and result download. A provider sees node enrollment instructions, agent token status, detected hardware, availability controls, active jobs and local utilization graphs. The administrator sees user/node approval, image allow-list, scheduler policy, queue depth, failed attempts, security events and audit search. Status will use both text and color, destructive actions will require confirmation, and validation errors will identify the exact unsupported field.

Interface	Required information and actions
Job submission	Image digest, command profile, CPU, RAM, storage, GPU/VRAM, timeout and priority.
Monitoring	State timeline, selected node alias, progress/log stream, attempt number and timestamps.
Node view	Hardware inventory, heartbeat age, free/reserved resources, utilization and drain switch.
Administration	Approvals, policy weights, trusted images, retry limits, audit events and health alerts.

3.2 Software Interface

The browser communicates with a versioned REST API. The API validates requests, persists the job and publishes a queue event. The scheduler reads jobs and recent resource snapshots, creates an allocation in a database transaction, then sends a short-lived signed command to the node agent. The agent is the only component allowed to call the local Docker Engine; the Docker socket is never exposed to a workload container. GPU jobs pass a device request to the NVIDIA runtime. Results are uploaded to object storage or a controlled shared directory, while PostgreSQL stores metadata and Redis carries transient queue/notification data.

Connection	Service and communication nature
Frontend -> API	JSON over HTTPS; bearer access token; server-sent status updates or secure WebSocket.
API -> PostgreSQL	Parameterized SQL through ORM/driver; transactions for job and allocation state.
Scheduler -> Redis	Durable queue/stream entry with acknowledgement and retry policy.
Control plane -> Agent	Mutual-authenticated TLS channel for heartbeat, reservation, start, cancel and result status.
Agent -> Docker/NVIDIA	Local Unix socket and NVIDIA Container Toolkit; no remote daemon exposure.

3.3 Database Interface

PostgreSQL is the system of record because job scheduling requires consistent state changes, relational integrity and recoverable transactions. An allocation is created only if the selected node still has enough unreserved capacity; the transaction locks the relevant node/resource row, writes the allocation and changes the job state together. PostgreSQL's default Read Committed level is sufficient for ordinary views, while the reservation path will use explicit row locking and retry on conflict [11]. Redis contains transient queue, cache and pub/sub data; it is not authoritative for final job state. Output files are stored outside the relational database and referenced by checksum, size and URI.

Table	Key fields	Purpose
users	user_id PK, email, password_hash, role, status	Owns jobs; approves or contributes nodes.
compute_nodes	node_id PK, provider_id FK, status, labels, last_seen	Stable node identity and lifecycle.
resource_snapshots	snapshot_id PK, node_id FK, CPU/RAM/GPU/disk/RTT, sampled_at	Time-series scheduling telemetry.
jobs	job_id PK, owner_id FK, image_digest, state, priority, timeout	Logical workload and requested policy.
job_requirements	job_id PK/FK, cpu, ram_mb, disk_mb, gpu_count, vram_mb	Hard resource constraints.
allocations	allocation_id PK, job_id FK, node_id FK, score, attempt, state	Reservation and scheduling evidence.
executions	execution_id PK, allocation_id FK, container_id, exit_code, timestamps	Actual container attempt.
audit_logs	event_id PK, actor_id, action, target, outcome, created_at	Append-only accountability record.

3.4 Protocols

Protocol area	Requirement
HTTPS / REST	TLS 1.3 preferred; JSON; request ID; API version; size limits; 10-30 s API timeout.
Secure WebSocket	Outbound agent connection for heartbeat and commands; ping/pong, sequence number and reconnect back-off.
Registry / artifacts	HTTPS with content digest and authorization; resumable uploads for larger results.
Time and synchronization	NTP; server timestamps are authoritative; snapshot freshness checked before scheduling.
Queue semantics	At-least-once delivery with idempotency key; agent rejects duplicate execution identifiers.

Heartbeat messages should remain below 5 KB and ordinary control messages below 64 KB. Large images and results travel directly between worker and registry/storage rather than through the scheduler. Retries use exponential back-off with jitter. Every command includes a monotonically increasing attempt number and expiry time to prevent a delayed message from starting an obsolete container.

4 NON-FUNCTIONAL REQUIREMENTS

4.1 Performance Requirements

Measure	Acceptance target
API responsiveness	95% of ordinary metadata requests complete within 500 ms under 50 concurrent browser sessions.
Scheduling latency	A queued job is evaluated within 2 s of arrival or an eligible-node update; decision computation below 250 ms for 100 nodes.
Telemetry freshness	Five-second samples; 10-second heartbeats; stale data excluded after 30 s.
Dispatch	Node acknowledges reservation within 5 s and reports container creation within 30 s, excluding image download.
Recovery	Lost node detected within 30-60 s; retryable job returned to the queue within 10 s of confirmed loss.
Prototype capacity	At least 100 registered nodes, 25 concurrent executions and 1,000 retained jobs without redesign.

The proposed policy will be compared with random and round-robin scheduling on the same trace. Reports will include median and p95 completion/queue time, CPU/GPU utilization, success rate, recovery delay and the tested hardware/network conditions.

4.2 Security Requirements

- Use Argon2id password hashing, short-lived signed tokens, refresh rotation and role-based authorization on every protected resource.
- Require administrator-approved node enrollment, renewable machine credentials and encryption for all external traffic.
- Run only allow-listed image digests; reject privileged containers and arbitrary host mounts; scan or verify images where tooling is available.
- Apply a non-root user, dropped capabilities, no-new-privileges, read-only filesystem where possible, seccomp/AppArmor and CPU/memory/PID/time/egress limits [9][10].
- Protect secrets at run time; owner-scope and checksum results; redact logs; audit authentication, policy, scheduling, execution and artifact access.

4.3 Software Quality Attributes

Attribute	Design response
Adaptability / flexibility	Scheduler weights, retry policy and node labels are configurable rather than hard-coded.
Availability / reliability / robustness	Health checks, durable state, idempotent commands, bounded retries and failure isolation address node churn.
Correctness / testability	Validation, resource invariants, transactional reservations, state-machine tests and failure injection prevent over-allocation.
Interoperability / portability / reusability	REST/JSON and OCI images; standard Linux workers; reusable runtime and storage adapters.
Maintainability / usability	Separated modules, typed contracts, migrations, accessible screens and actionable failure messages.

5 OTHER REQUIREMENTS

The repository will include deployment instructions, environment-variable samples, API documentation, database migrations, scheduler test cases and a repeatable demonstration script. The prototype will use open-source components whose licences permit academic use. Operators must obtain consent before enrolling institutional hardware and must comply with laboratory rules, software licences and data-handling policy. Accessibility checks, log retention and backup/restore tests are part of the completion criteria. The final demonstration will show node enrollment, one CPU job, one GPU job, an explainable scheduling decision and failure-triggered requeueing.

Appendix A: Glossary

Term	Meaning in this project
Agent	Service on a provider machine that reports resources and controls local containers.
Allocation	A time-bounded reservation connecting one job attempt to one compute node.
Control plane	API, scheduler, registry, queue and persistence services that coordinate workers.
DRF	Dominant Resource Fairness; a method for fair sharing across multiple resource types.
Heartbeat	Periodic liveness and summary message from a node agent.
OCI image	Portable container-image format identified by an immutable digest.
Telemetry	Time-stamped CPU, RAM, GPU, storage, network and execution observations.

Appendix B: Analysis Model

The evaluation uses a repeated workload trace containing CPU-only, GPU-required and mixed jobs, stratified by short/long duration and small/large memory request. The identical trace is replayed under random, round-robin and proposed resource-aware policies. For each job j, turnaround Tj = completionj - submissionj and waiting Wj = startj - submissionj. Cluster utilization is the time-weighted used capacity divided by available capacity. Success rate is completed jobs divided by admitted jobs; recovery delay is the interval from the last valid heartbeat to reassignment. Medians, p95 values and paired bootstrap confidence intervals will be reported, together with scheduling logs that explain why a node was selected.

Appendix C: Issues List

ID	Issue	Status	Planned resolution
I-01	GPU sharing mode	Open	Begin with whole-device allocation; assess MIG/time-slicing only on compatible hardware.
I-02	Checkpoint support	Open	Recovery restarts jobs; define an optional application-level checkpoint contract.
I-03	Image trust	Open	Finalize registry allow-list, digest pinning and signature-verification workflow.
I-04	Artifact storage	Open	Choose MinIO/S3-compatible storage or controlled shared directory for the lab.
I-05	Policy weights	Open	Tune weights using pilot runs; publish defaults and sensitivity results.
I-06	Control-plane outage	Deferred	Prototype uses one server; document backup/restore before adding high availability.